

APARA: A 1-TOPS Edge-compute Accelerator

Madhav Desai
Department of Electrical Engineering
IIT Bombay

July 19, 2026

Contents

1	Overview of the APARA accelerator	1
2	The execution pipeline and the multi-threading mechanism	3
2.1	Thread state and thread registers	3
3	Data types	5
4	Instruction Bundles	7
4.1	Bundle start bit in each instruction	8
5	Arithmetic and logic instructions	9
5.1	Scalar arithmetic logic instructions	9
5.2	Float square-root instruction	10
5.3	Absolute-value instruction (in progress)	11
5.4	Vector arithmetic instructions	11
5.5	Vector DOT instruction	12
5.6	Vector REDUCE instruction	13
6	Control transfer instructions	14
6.1	BRANCH instruction	14
6.2	CALL instruction	15
6.3	RETURN instruction	16
6.4	CMOV instruction	16
7	Data transfer instructions	18
7.1	LOAD instruction	18
7.2	STORE instruction	19

8	Miscellaneous instructions	22
8.1	SET instruction	22
8.2	SLICE instruction	23
8.3	CAST instruction	23
8.4	PACK instruction	25
8.5	NULL, NOP, HALT Instructions	25
9	Sample Program: i32 dot product of 1024 element vectors	27
10	The APARA assembler and related tools	31
11	System architecture scenarios of the use of APARA	32
12	Quirks	34
12.1	Memory accesses are aligned to natural byte boundaries	34
12.2	Register accesses are aligned to natural indices	34
12.3	Control transfer only in full bundles	34
12.4	Memory access, divide/square-root instructions only in full bundles	35
12.5	Limits on memory-access, control-transfer, divide/square-root instructions in an 8-bundle	35

Abstract

We describe the instruction set architecture of the APARA edge compute accelerator. The accelerator is a multi-threaded compute engine, with four threads multiplexed onto a single execution pipeline. The instruction and data memories for each thread are disjoint. Each thread executes an instruction stream which consists of a sequence of bundles which can have 1/2/4/8 instructions which are executed in parallel. The instruction set itself consists of various classes: integer/float ALU, load/store, branch/call/return, and others. Types ranging from 4-bit (int/float) to 64-bit (int/float) are supported. Data transfer instructions can transfer between 1 and 32 bytes between the pipeline and attached data memory.

Chapter 1

Overview of the APARA accelerator

We illustrate the APARA edge compute accelerator in Figure 1. The core of the accelerator is the execution pipeline, which can simultaneously execute up to 8 instructions. Four execution threads are multi-plexed onto this execution pipeline. Each thread has its own register set, return address stack and control/status registers which can be accessed through the control interface.

The control interface is expected to hook up with a configuration/management processor which uses the control interface to

- Program the instruction memory of the execution pipeline.
- Program/Observe the registers corresponding to each execution thread.
- Schedule actions in the accelerator.
- Schedule and monitor DMA operations between the accelerator memory and the system memory.

The DMA engine provides data transfer between the system memory and the accelerator data memory. The DMA engine can be programmed to copy bursts of data from one point to another. Using this, it is possible to extract/insert from/into matrices and tensors from one memory area to another. For example, it is possible to extract a 16x16 submatrix of a larger matrix and keep the slice in the accelerator data memory etc.

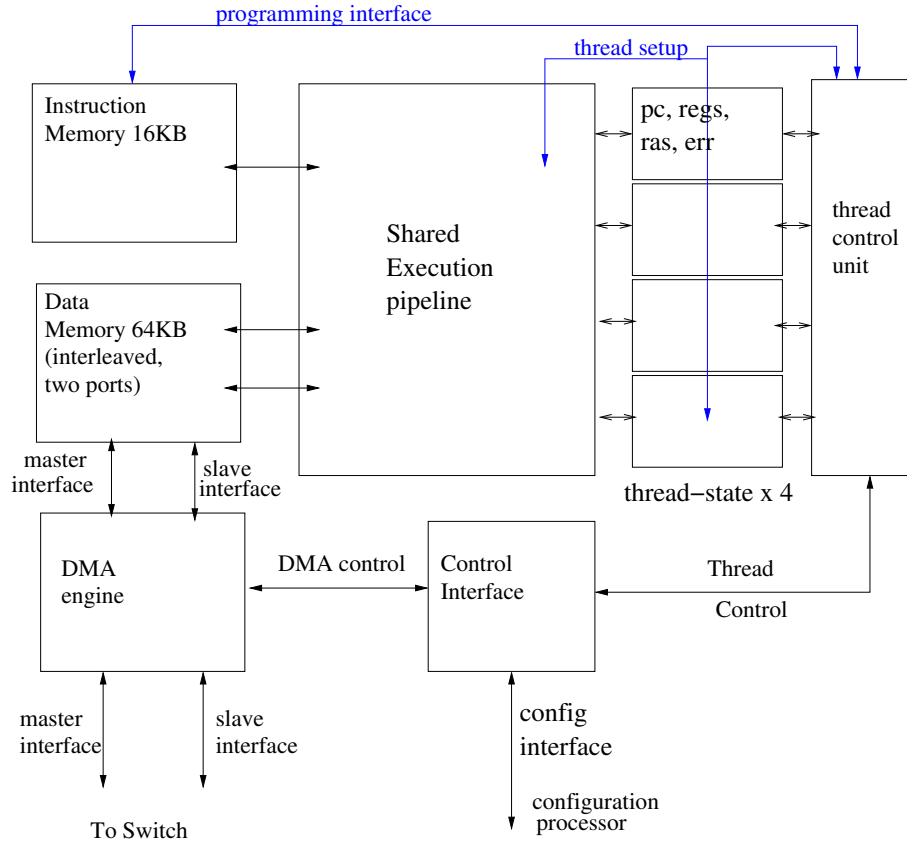


Figure 1.1: Register and memory specifications

The instruction memory provides 16KB of instruction space to each accelerator. Each instruction is 4-bytes. Instructions are accessed by the execution pipeline in chunks up to 32-bytes in size (that is, up to 8 instructions at a time can be fetched). The instruction set will be described later.

The data memory provides 64KB of space for use by the execution pipeline. The data memory has two banks, each with 32KB of memory. It is possible to access 32-bytes per cycle at a time from each of the banks. Thus, the maximum data bandwidth available to the execution pipeline is 64Bytes per cycle.

Chapter 2

The execution pipeline and the multi-threading mechanism

The threading mechanism and its interaction with the execution pipeline can be illustrated by the algorithm shown in Figure 2.1. Each thread has its own internal state, consisting of the program counter (PC), a register set with 32 64-bit registers, a return address stack and an error register.

The execution pipeline interleaves fetches from the main memory to generate an interleaved instruction stream. A thread which has an instruction available for execution is picked, and the instruction corresponding to the selected thread is executed.

2.1 Thread state and thread registers

The thread state consists of a 32-bit program counter, a 32-bit error code register, and 32 64-bit general purpose registers. The register with index 0 is always read as 0.

```

// 4 threads, 4x PC, 4x REGSETS, 4x RAS, 4x ERR
PC[1 to 4] = initvalues , REGS[1 to 4],
RAS[1 to 4] = empty, ERR[1 to 4] = 0, ACTIVE=1

while(1)
{
    bundle := fetch_bundle_from_imem(PC[ACTIVE])
    ops    :=
        get_ops_needed_by_bundle(bundle, REGS[ACTIVE])
    out_values :=
        execute_instructions_in_bundle(bundle, ops)
    PC[ACTIVE], REGS[ACTIVE], ERR[ACTIVE] :=
        update_registers_pc_ras_error (out_values)
    ACTIVE = next_active (ACTIVE, ERR[1 to 4]);

    if(ACTIVE==0) break;
}

```

Figure 2.1: Multi-thread execution algorithm

Chapter 3

Data types

The contents of a register can be viewed as data with the following data types

type	symbol	description
\$i4		4-bit signed integer
\$i8		8-bit signed integer
\$i16		16-bit signed integer
\$i32		32-bit signed integer
\$i64		64-bit signed integer
\$u4		4-bit unsigned integer
\$u8		8-bit unsigned integer
\$u16		16-bit unsigned integer
\$u32		32-bit unsigned integer
\$u64		64-bit unsigned integer
\$u128		128-bit unsigned integer (across two registers)
\$u256		256-bit unsigned integer (across four registers)
\$f4		4-bit float
\$f8		8-bit float

\$f16	16-bit float
\$f32	32-bit float
\$f64	64-bit float
\$vi4	vector of 4-bit signed integer
\$vi8	vector of 8-bit signed integer
\$vi16	vector of 16-bit signed integer
\$vi32	vector of 32-bit signed integer
\$vu4	vector of 4-bit unsigned integer
\$vu8	vector of 8-bit unsigned integer
\$vu16	vector of 16-bit unsigned integer
\$vu32	vector of 32-bit unsigned integer
\$vf4	vector of 4-bit float
\$vf8	vector of 8-bit float
\$vf16	vector of 16-bit float
\$vf32	vector of 32-bit float

Note: vector types with 4-bit elements will
be viewed as 16x4 etc..

Chapter 4

Instruction Bundles

Each instruction is 32-bits wide. Instructions are organized into bundles of 1/2/4/8 instructions. Each bundle has a start instruction address, and each instruction inside a bundle has an instruction address.

Bundle start instruction addresses are required to be aligned. That is, the start address of a bundle with 8-instructions is a multiple of 8 and the start address of a bundle with 4-instructions is a multiple of 4 and so on.

The instructions fall into the following classes:

- Scalar arithmetic-logic instructions.
 - Binary arithmetic, logic operations.
 - Unary arithmetic (absolute-value, floating point square-root).
- Vector arithmetic instructions.
 - Vector add/subtract/multiply.
 - Dot-product instruction.
 - Vector-reduce instruction.
- Data transfer instructions (load/store).
- Control transfer instructions (branch/call/return/cmov).
- Miscellaneous instructions
 - set, slice, pack, null, nop, halt instructions.

We will describe these instructions now. The description will be in an assembly code format. In this assembly code format, registers will be referred to as

`$r0, $r1, ... $r31`

`$r0` always reads as 0.

Initially, `$r1,$r2, .. $rn` have random contents.

4.1 Bundle start bit in each instruction

Each instruction is 32-bits wide. The most significant bit of the instruction is called the bundle-start bit. The start of a bundle is indicated by this bit being set.

The op-code for each instruction always occupies bits 30:26 of the instruction word.

Chapter 5

Arithmetic and logic instructions

These fall into four classes: scalar ALU, square-root, vector arithmetic, dot-product and vector-reduce.

5.1 Scalar arithmetic logic instructions

Scalar arithmetic logic instructions have one of the following forms. The first one is

`<op-code> <destination-reg> <src-type> <src-register-1> <src-register-2>`

e.g.

`+ $r3 ($i32) $r1 $r2`

means: the contents of \$r1 and \$r2 are interpreted as 32-bit numbers, added and their result is saved as 64-bit signed integer.

The second form is

`<op-code> <destination-reg> <src-type> <src-register-1> <constant-value>`

e.g.

+ \$r3 (\$i32) \$r1 256

Instead of the second register, a constant value (which is treated as a 10-bit signed integer) is provided.

Here, the opcode can be one of the following (each opcode is represented by 5-bits)

mnemonic	bits	description
+	0x4	add
-	0x5	sub
*	0x6	mul
/	0x7	div
	0x8	or
&	0x9	and
^	0xA	xor
~	0xB	nor
~&	0xC	nand
^^	0xD	xnor
<<	0xE	shift-left
>>	0xF	shift-right
\$max	0x1C	max-of-two (in progress)

The binary coding for these instructions is as follows. For the first form:

1	30:26	25:21	20:16	15:11	10:10	9:5	4:0
bstart	opcode	type	rd	rs1	0	0	rs2

For the second form:

31	30:26	25:21	20:16	15:11	10:10	9:0
bstart	opcode	type	rd	rs1	1	imm-value

5.2 Float square-root instruction

The assembly format for the instruction is

`$fsqrt <destination-reg> <src-type> <src-register-1>`

e.g.

`$fsqrt $r3 ($f16) $r1`

calculates a 16-bit floating point square-root
and writes it into \$r3.

The binary format for this instruction is

31	30:26	25:21	20:16	15:11	10:0
bstart	0x10	type	rd	rs1	unused

5.3 Absolute-value instruction (in progress)

The assembly format for the instruction is

`$abs <destination-reg> <src-type> <src-register-1>`

e.g.

`$abs $r3 ($f16) $r1`

calculates a 16-bit floating point absolute-value of
\$r1 and writes it into \$r3.

The binary format for this instruction is

31	30:26	25:21	20:16	15:11	10:0
bstart	0x1c	type	rd	rs1	unused

5.4 Vector arithmetic instructions

These have a similar format to the scalar arithmetic logic instructions described earlier. Only add/sub/mul operations are performed on vectors. The immediate form is absent, and an additional replicate flag is included.

`$v <op-code> <destination-reg> <src-type> <src-register-1>\`
`<src-register-2> $replicate?`
opcode is + or - or *

e.g.

```
$v + $r3 ($vi32) $r1 $r2
$v + $r3 ($vi32) $r1 $r2 $replicate
```

The first example means: the contents of \$r1 and \$r2 are interpreted as two two-element vectors which are added element-wise and the result is stored in \$r3.

The second example means: the 32-bit scalar in \$r2 is replicated and the resulting vector is added to the vector in \$r1.

The binary format of these instructions is

1	31:26	25:21	20:16	15:11	10:10	9:5	4:0
bstart	opcode	type	rd	rs1	replicate	0	rs2

5.5 Vector DOT instruction

These instructions specify a DOT product of two vectors with an optional accumulation. **The dot instruction is defined only for elements whose type is 16-bits wide or less.**

The assembly form of the dot instruction is

```
$dot <rd> <src-type> rs1 rs2 $accumulate? $replicate?
```

examples

```
$dot $r3 ($vi4) $rs1 $rs2
$dot $accumulate $r3 ($vi4) $rs1 $rs2
$dot $r3 ($vi4) $rs1 $rs2 $replicate
$dot $accumulate $r3 ($vi4) $rs1 $rs2 $replicate
```

For integer element types, the result of the dot is kept as 64-bit integer. For float element types, the result of the dot is kept as a 32-bit float.

The accumulate form calculates

```
rd := (r1 dot r2) + rd
```

The non-accumulate form calculates


```
rd := (r1 dot r2)
```

The replicate form calculates

```
rd := (r1 dot replicate(r2))
```

The accumulate and replicate form calculates

```
rd := (r1 dot replicate(r2)) + rd
```

The binary format for the dot instruction is

[31]	30:26	25:21	20:16	15:11	10:10	9:9	8:5	4:0
bstart	0x11	type	rd	rs1	accumulate	replicate	0	rs2

5.6 Vector REDUCE instruction

The reduce instruction takes a vector of elements and sums them up. The assembly description of the instruction is

```
$vreduce <op-code> $rd <src-type> $rs1
```

examples

```
$vreduce + $r2 ($vi4) $r1
```

```
$vreduce * $r2 ($vi8) $r1
```

The first example sums up the 4-bit values in the vector stored in \$r1 and saves the resulting sum in \$r2 as a 64-bit integer (signed/unsigned, depending on the source type's signed-ness). The second example uses the multiply operation on \$i8 elements to perform the reduction.

The binary format for the reduce instruction is

[31]	[30:26]	[25:21]	[20:16]	[15:11]	[10:6]	[5:0]
bstart	0x16	type	rd	rs1	sub-op-code	unused

The supported sub-op-codes (max/min in progress) are

```
+, -, *, &, |, ^, ^^, $max, $min
```

Chapter 6

Control transfer instructions

Control transfer instructions are used to transfer control to a bundle address. There can be at most one control transfer instruction present in a bundle. Further, the bundle containing the control transfer instruction must have eight instructions in it. This is enforced by the `mcode_align` utility.

There are three types of control transfer instructions: branch, call, return.

6.1 BRANCH instruction

A branch instruction specified a branch register, a branch condition and a PC-relative address jump. The contents of the branch register are used to compute the branch condition and the branch either redirects the control flow to the branch destination or continues to the default next-instruction.

The assembly code for a branch can be of the following three forms

```
? comparison-type register condition auto-increment goto label
```

For example:

```
? ($i16) $r5 > $goto bundle_label
? ($i16) $r5 > --1 $goto bundle_label
? ($i16) $r5 < ++1 $goto bundle_label
```

Comparison conditions are always with zero. The following conditions are recognized:

```
-----
condition description
```

```

-----
==          equal-to-0
>          > 0
<          < 0
>=         >= 0
<=         <= 0
!=         != 0
-----

```

The binary code for a branch instruction has the format

```

[31]  [30:26] [25:22] [21:19]  [18:17]  [16:5] [4:0]
bstart 0x2      cmp   condition incr/decr  jump   rs2
                        type

```

The 3-bit code for the condition is

```

-----
3-bit code  condition
-----
001         == 0
010         > 0
011         < 0
100         >= 0
101         <= 0
110         != 0
-----

```

6.2 CALL instruction

The CALL instruction specifies a jump to a 25-bit PC-relative offset. The return address is the call instruction program-counter + 1. The return address is stored on a return address stack (RAS) which is 64 deep. Overflow or underflow of the RAS is considered a fatal error.

The assembly description of the CALL instruction has two forms.

```

$call <label>
$call <register-id>

```

In the first form, the target address is that specified by a bundle label. In the second form, the target address is in the bottom 32-bits of the specified register id.

The binary code for the CALL instruction can be one of

```
[31]    [30:26] [25] [24:0]
bstart  0x1      imm  relative jump
                        (=1)
```

```
[31]    [30:26] [25] [24:5]    [4:0]
bstart  0x1      imm  unused    regid
                        (=0)
```

6.3 RETURN instruction

The RETURN instruction causes a jump to the address at the top of the return address stack (RAS). The RAS is then popped. Overflow/underflow of the RAS causes the program to terminate.

The assembly description of the RETURN instruction is

```
$return
```

The binary code for the RETURN instruction is

```
[31]    [30:26] [25:0]
bstart  0x3      unused
```

6.4 CMOV instruction

The CMOV (conditional-move) instruction allows us to alter the data-flow in a program. A check register and a condition are specified. If the contents of the check register meet the condition, then data is copied from a specified source register into the specified destination register. An auto-increment on the check register can also be specified.

The assembly code description of the CMOV instruction has several forms, similar to the BRANCH instruction.

```
$cmov ($i8) <rs1> > --1 <rd> <rs2>
```

for example

```
$cmov ($i8) $r1 > --1 $r3 $r2
```

The interpretation: if the contents of the register \$r1 (interpreted as an 8-bit signed integer) meet the condition specified, then the contents of \$r2 are copied into \$r3. Otherwise the contents of \$r3 are left unchanged. The auto-decrement is then applied to \$r1.

Without a specified auto-decrement, we would write

```
$cmov ($i8) $r1 > $r3 $r2
```

The binary code of the CMOV instruction is

31	30:26	25:21	20:16	15:11	10:8	7:6	5	4:0
bstart	0x17	type	rd	rs1	cond	incr	unused	rs2

The three bits of the condition are interpreted in the same way as for the branch instruction.

Chapter 7

Data transfer instructions

Data transfer instructions can be loads or stores. The number of bytes transferred can be 1/2/4/8/16/32. Data is moved between memory and registers. Up to 4 registers can provide the data for the store operation and consume the data from the load operation.

7.1 LOAD instruction

The LOAD instruction specifies a memory address (via register indexed addressing) and the registers where the data from the memory is to be received.

The memory address is computed using either

`<rs1> + <rs2>`

or

`<rs1> + constant`

The computed address is a byte address and must be aligned to the transfer size. That is a byte address can be any integer, a 2-byte address must be a multiple of 2 etc.

The assembly code description of the STORE instruction can be of two types.

```
$ld <type> <rd> [<rs1> + <rs2>]  
$ld <type> <rd> [<rs1> + <imm-value>]
```

For example:

```
$ld ($u8) $r3 [$r1 + $r2 ]  
$ld ($u8) $r3 [$r1 + 16  ]
```

The immediate value is a 10-bit signed integer.

It is also possible to indicate address auto-increment/decrement in the LOAD instruction

```
$ld ($u8)  $r3 [$r1 + $r2 ++1]  
$ld ($u16) $r3 [$r1 + $r2 ++1]
```

The increment quantity is multiplied by the transfer length in bytes and then added to \$r2 in the example above. In the first example above, \$r2 is incremented by 1, and in the second, by 2.

The binary code for the LOAD instruction is one of:

[31]	[30:26]	[25:21]	[20:16]	[15:11]	[10]	[9:5]	[4:0]
bstart	0x13	type	rd	rs1	imm	incr	rs2
						=0	

[31]	[30:26]	[25:21]	[20:16]	[15:11]	[10]	[9:0]
bstart	0x13	type	rd	rs1	imm	imm-value
					=1	10-bit signed

When the transfer size is 128-bits, two source registers are used and the specified source register must have an even index.

When the transfer size is 256 bits, four source registers are used and the specified source register must have an index which is a multiple of 4.

7.2 STORE instruction

The STORE instruction specifies a memory address (via register indexed addressing) and the registers from which the data to be written to the memory is to be taken.

The memory address is computed using either

<rs1> + <rs2>

or

<rs1> + constant

The computed address is a byte address and must be aligned to the transfer size. That is a byte address can be any integer, a 2-byte address must be a multiple of 2 etc.

The assembly code description of the STORE instruction can be of two types.

```
$st <type> [<rs1> + <rs2>] <rd>
$st <type> [<rs1> + <imm-value>] <rd>
```

For example:

```
$st ($u8) [$r1 + $r2] $r3
$st ($u8) [$r1 + 16] $r3
```

The immediate value is a 10-bit signed integer.

It is also possible to indicate address auto-increment/decrement in the LOAD instruction

```
$st ($u8) [$r1 + $r2 ++1] $r3
$st ($u16) [$r1 + $r2 ++1] $r3
```

The increment quantity is multiplied by the transfer length in bytes and then added to \$r2 in the example above. In the first example above, \$r2 is incremented by 1, and in the second, by 2.

The binary code for the STORE instruction is one of:

31	[30:26]	[25:21]	[20:16]	[15:11]	[10]	[9:5]	[4:0]
bstart	0x14	type	rd	rs1	imm	incr	rs2
					=0		

or

31	[30:26]	[25:21]	[20:16]	[15:11]	[10]	[9:0]
----	---------	---------	---------	---------	------	-------

bstart	0x14	type	rd	rs1	imm	imm-value
					=1	10-bit signed

When the transfer byte size is 128, four destination registers are used and the specified destination register must have an index which is a multiple of 2.

When the transfer byte size is 256, four destination registers are used and the specified destination register must have an index which is a multiple of 4.

Chapter 8

Miscellaneous instructions

These are miscellaneous bit manipulation instructions.

8.1 SET instruction

SET allows you to set the bottom 16 bits of a register with a constant value.

`$set <rd> <low-byte-index> const-value`

for example

`$set $r5 0 16`

`$set $r5 1 16`

`$set $r5 2 16`

In the first example above, the value 16 is copied into `$r5[15:0]`. In the second example, the value 16 is copied into `$r5[23:8]`. In the third example, the value 16 is copied into `$r5[31:16]`. Thus it is possible to write a 16-bit value into one of many positions in the destination register.

Note that

`$set $r5 7 16`

will copy the lower byte of 16 into `$r5[63:54]` and the upper byte of 16 is thrown away.

The binary code for the SET instruction is

31	[30:26]	[25:24]	[23:21]	[20:16]	[15:0]
bstart	0x15	unused	dest	rd	16-bit
			lowbyte		constant

8.2 SLICE instruction

The SLICE instruction extracts a section out of a source register and places that section in a destination register.

The assembly code description for this instruction is

```
$slice <rd> <hindex> <lowindex> <rs>
```

for example

```
$slice $r2 20 7 $r1
```

For the instruction in the example, the destination \$rd is set to \$r1[20:7] (in fact \$rd[13:0] is set to \$r1[20:7], and \$rd[63:14] is set to 0).

The binary code for the slice instruction is

[31]	[30:26]	[25:21]	[20:16]	[15]	[14:9]	[8:5]	[4:0]
bstart	0x19	hindex	rd	hindex	lowindex	unused	rs2
		[5:1]		[0]			

8.3 CAST instruction

The CAST instruction plays a very important role in the thread instruction set. Type conversions of the following kind are supported.

- Scalar integer to integer type conversions.
- Scalar float to float type conversions.
- Scalar float to integer type conversions.
- Scalar integer to float type conversions.
- Vector integer to vector integer type conversions.
- Vector float to vector float type conversions.
- Vector float to vector integer type conversions.
- Vector integer to vector float type conversions.

Examples of each of the type conversions listed above are

`$cast <dest-type> <dest-reg> <src-type> <src-reg>`

Examples: scalar casts

```
$cast ($i8) $r2 ($i16) $r1 // scalar int->int
$cast ($u8) $r2 ($i16) $r1 // scalar int->uint
$cast ($i8) $r2 ($u16) $r1 // scalar uint->int
$cast ($f8) $r2 ($u16) $r1 // scalar uint->float
$cast ($u8) $r2 ($f16) $r1 // scalar float->uint
$cast ($f8) $r2 ($f16) $r1 // scalar float->float
```

Examples: vector casts

```
$cast ($vi16) $r3 ($vf16) $r1
$cast ($vf8) $r3 ($vf16) $r1
$cast ($vi16) $r4 ($vi8) $r1
```

etc.

Only the following four use scenarios are permitted for the CAST instruction:

- For vector casts, the source vector element type can have the same width as the destination vector element type. In this case, there is one source register (\$r1 in the example above) and one destination register (\$r2 in the example above).
- For vector casts, the source vector element type can have twice the width of the destination vector element type. In this case, two source vectors (\$r1, \$r2) are used to generate a single destination register \$r3. **Each of the registers \$r1 and \$r2 must have even indices in this case.**
- For vector casts, the destination vector element type can have twice the width of the source vector element type. In this case, the single source register \$r1 produces two destination registers \$r4, \$r5. **when there is a pair of registers implied as a destination, then the specified register must have an even index.**
- For scalar casts, a single source register produces a single destination register.

The binary code for the CAST instruction is

[31]	[30:26]	[25:21]	[20:16]	[15:11]	[10:5]	[4:0]
bstart	0x12	dest-	rd	src-	0	rs2
		type		type		

8.4 PACK instruction

The PACK instruction is used to collect data from two registers and pack that data into a single register.

For example

```
$pack <rd> <packed-result-nbits> <per-source-nbits> <rs2>
```

For example:

```
$pack $r5 32 16 $r1
```

Sixteen bit values from two source registers \$r1, \$r2, are collected and packed into \$r5. The contents of \$r5[31:16] are filled with \$r1[15:0], and the contents of \$r5[15:0] are filled with \$r2[15:0].

Note that at most two registers can be packed into a destination. A trivial pack is

```
$pack $r5 64 64 $r1
```

where \$r1 is simply copied into \$r5.

The binary code for the PACK instruction is

[31]	[30:26]	[25:21]	[20:16]	[15:12]	[11:9]	[8:5]	[4:0]
bstart	0x18	0	rd	nbits	nbits	0	rs2
				in dest	src		

When the destination register is a pair, the index of the specified register must be even.

8.5 NULL, NOP, HALT Instructions

The NULL and NOP instructions are no-ops. The HALT instruction terminates a running thread.

The assembly description of these three instructions is

\$null
\$nop
\$halt

The binary codes of these three instructions are

```
// nop
[31]      [30:26] [25:0]
bstart    0x0     0

//null
[31]      [30:26] [25:0]
bstart    0x1a    0

//halt
[31]      [30:26] [25:0]
bstart    0x1b    0
```

Chapter 9

Sample Program: i32 dot product of 1024 element vectors

Here is an example of a program which takes two vectors with 1024 elements of type i32, and produces a dot product of these two vectors. The base pointers of the two vectors are assumed to be in registers \$r1 and \$r2 respectively. The address of the location where the result is to be written is in \$r5. We use \$r3 to keep a counter which initially has value 127. The final result is written into the memory location whose address is in \$r5.

```
init1:
||
| $r24 ($i64) $r0 $r0    // init r24 to 0.
| $r25 ($i64) $r0 $r0    // init r25 to 0.
| $r26 ($i64) $r0 $r0    // init r26 to 0.
| $r27 ($i64) $r0 $r0    // init r27 to 0.
| $r28 ($i64) $r0 $r0    // init r28 to 0.
| $r29 ($i64) $r0 $r0    // init r29 to 0.
| $r30 ($i64) $r0 $r0    // init r30 to 0.
| $r31 ($i64) $r0 $r0    // init r31 to 0.
;
init2:
||
$set $r3 0 127    // r3 to 127
$set $r1 0 0
$set $r2 0 4096
```

```

    | $r4 ($i64) $r0 $r0    // init r4 to 0.
    + $r5 ($i64) $r0 32    // init r5 to 32
;
mem:    // fetch 8 + 8 regs from mem.
||
    $ld ($u256) $r8    [$r1 + $r4 ++2]
    $ld ($u256) $r12   [$r2 + $r4]
    $ld ($u256) $r16   [$r1 + $r5 ++2]
    $ld ($u256) $r20   [$r2 + $r5]
;
dot1:   // vector products (8 of them, ie 16 ops)
||
    $v * $r8 ($vi32) $r8 $r12
    $v * $r9 ($vi32) $r9 $r13
    $v * $r10 ($vi32) $r10 $r14
    $v * $r11 ($vi32) $r11 $r15
    $v * $r12 ($vi32) $r16 $r20
    $v * $r13 ($vi32) $r17 $r21
    $v * $r14 ($vi32) $r18 $r22
    $v * $r15 ($vi32) $r19 $r23
;
dot2:   // reduce sums (8 ops)
||
    $vreduce $r8 ($vi32) $r8
    $vreduce $r9 ($vi32) $r9
    $vreduce $r10 ($vi32) $r10
    $vreduce $r11 ($vi32) $r11

    $vreduce $r12 ($vi32) $r12
    $vreduce $r13 ($vi32) $r13
    $vreduce $r14 ($vi32) $r14
    $vreduce $r15 ($vi32) $r15

;
dot3:   // residual summation (8 ops)
||

    + $r24 ($i32) $r8 $r24

```



```

+ $r25 ($i32) $r9 $r25
+ $r26 ($i32) $r10 $r26
+ $r27 ($i32) $r11 $r27
+ $r28 ($i32) $r12 $r28
+ $r29 ($i32) $r13 $r29
+ $r30 ($i32) $r14 $r30
+ $r31 ($i32) $r15 $r31

;
loopback:      // 0 ops.
||
    $ld ($u256) $r8    [$r1 + $r4 ++2]
    $ld ($u256) $r12   [$r2 + $r4]
    $ld ($u256) $r16   [$r1 + $r5 ++2]
    $ld ($u256) $r20   [$r2 + $r5]

    ? ($i16) $r3 > --1 $goto dot1
;
finalsum1:
||
    + $r24 ($i32) $r24 $r25
    + $r25 ($i32) $r26 $r27
    + $r26 ($i32) $r28 $r29
    + $r27 ($i32) $r30 $r31
;
finalsum2:
||
    + $r24 ($i32) $r24 $r25
    + $r25 ($i32) $r26 $r27
;
finalsum3:
||
    + $r24 ($i32) $r24 $r25
;
wmem:      // write the result and halt.
||
    $st ($u32) [$r0 + 0] $r24
    $halt

```

;

Chapter 10

The APARA assembler and related tools

The APARA assembler family consists of the following utilities:

- **mcode_align** : Given an mcode program, the the align utility pads using null instructions to ensure that bundles with sizes 1/2/4/8 are aligned to 1/2/4/8 wide instruction word indices. The align utility also pads to ensure that control-transfer instructions are in bundles of size 8.
- **mcode_assemble** : Assembles an aligned mcode program and produces the executable binary code as a sequence of instruction words.
- **mcode_disassemble** : Dis-assembles an executable binary program back into assembly. Label id's are reconstructed, and original labels are not preserved.
- **mcode_run** : An mcode simulator which executes an mcode program. Initial values for data memory are provided, and so is a post-condition results file which the the simulator checks for correctness.

Chapter 11

System architecture scenarios of the use of APARA

The APARA accelerator is designed to be used in conjunction with a general purpose management processor (for example, the AJIT processor). In Figure 11, we show a potential system configuration with one management processor, one APARA accelerator instance, the system memory and peripherals.

The primary interactions between the management processor and the APARA instance are

- The management processor (MP) configures the APARA instance. This includes loading the instruction memory of the APARA instance and using the DMA engine in the APARA instance to transfer data to the embedded APARA memory.
- The MP initiates an execution thread in the APARA instance.
- The MP monitors the thread execution and on completion uses the DMA engine to transfer data from the embedded APARA memory to the system memory.
- Many of these activities are concurrent in nature, and the scheduling of these is to be done by management processor software.

It is possible to scale the system by increasing the number of management processors and APARA instances. Typically, we would scale the number of processors and APARA instances to four each. Further scaling is achieved by implementing a message passing cluster based system with high performance memory based message queues.

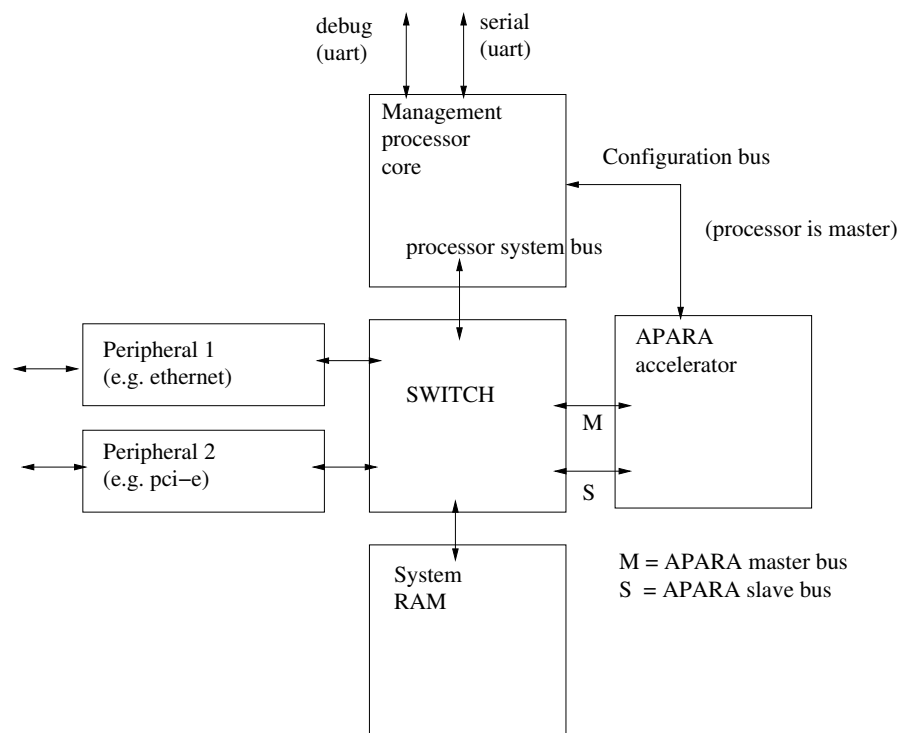


Figure 11.1: Potential single processor, single APARA instance system

Chapter 12

Quirks

Mcode can be viewed as micro-code and is tied to the hardware construction which imposes certain limitations.

12.1 Memory accesses are aligned to natural byte boundaries

Memory accesses are aligned to natural byte boundaries. Thus a double word access is to an address which is a multiple of 8 and so on.

12.2 Register accesses are aligned to natural indices

When a pair of registers is used as a source/destination the register specified in the instruction must have an even index.

When a quad of registers is used as a source/destination the register specified in the instruction must have an index which is a multiple of 4.

12.3 Control transfer only in full bundles

A control transfer instruction must be present only in bundles with 8 instructions. The aligner will enforce this during the assembly process.

12.4 Memory access, divide/square-root instructions only in full bundles

Divide/square-root, and memory access instructions must be part of full bundles with 8 instructions. The aligner will enforce this during the assembly process.

12.5 FP divide instructions do not support immediate flags

FP divide instructions are not permitted to have the immediate flag set. However, integer divide instructions do support the immediate flag.

12.6 Limits on memory-access, control-transfer, divide/square-root instructions in an 8-bundle

In a single 8-bundle, we can have at most 4 memory access instructions, 1 divide/square-root instruction and 1 control transfer instruction. The aligner will throw an error if these limits are exceeded.